

Lab-8.2

D.Himesh
2303A52368
Batch-38

Task-1

Test-Driven Development for Even/Odd Number Validator

- Use AI tools to first generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests.

Requirements:

- Input must be an integer
- Handle zero, negative numbers, and large integers

Example Test Scenarios:

`is_even(2)` → True
`is_even(7)` → False
`is_even(0)` → True
`is_even(-4)` → True
`is_even(9)` → False

Code:

```
import os
import sys
import unittest
```

```
# Ensure project root is on sys.path when running this test file directly
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
```

```
from even import is_even
```

```
class TestIsEven(unittest.TestCase):
    def test_even_positive(self):
        self.assertTrue(is_even(2))
```

```
    def test_odd_positive(self):
        self.assertFalse(is_even(7))
```

```
    def test_zero(self):
        self.assertTrue(is_even(0))
```

```
    def test_negative_even(self):
        self.assertTrue(is_even(-4))
```

```
    def test_negative_odd(self):
        self.assertFalse(is_even(-9))
```

```
    def test_large_numbers(self):
        self.assertTrue(is_even(10**18))
        self.assertFalse(is_even(10**18 + 1))
```

```
    def test_non_integer_raises(self):
        with self.assertRaises(TypeError):
```

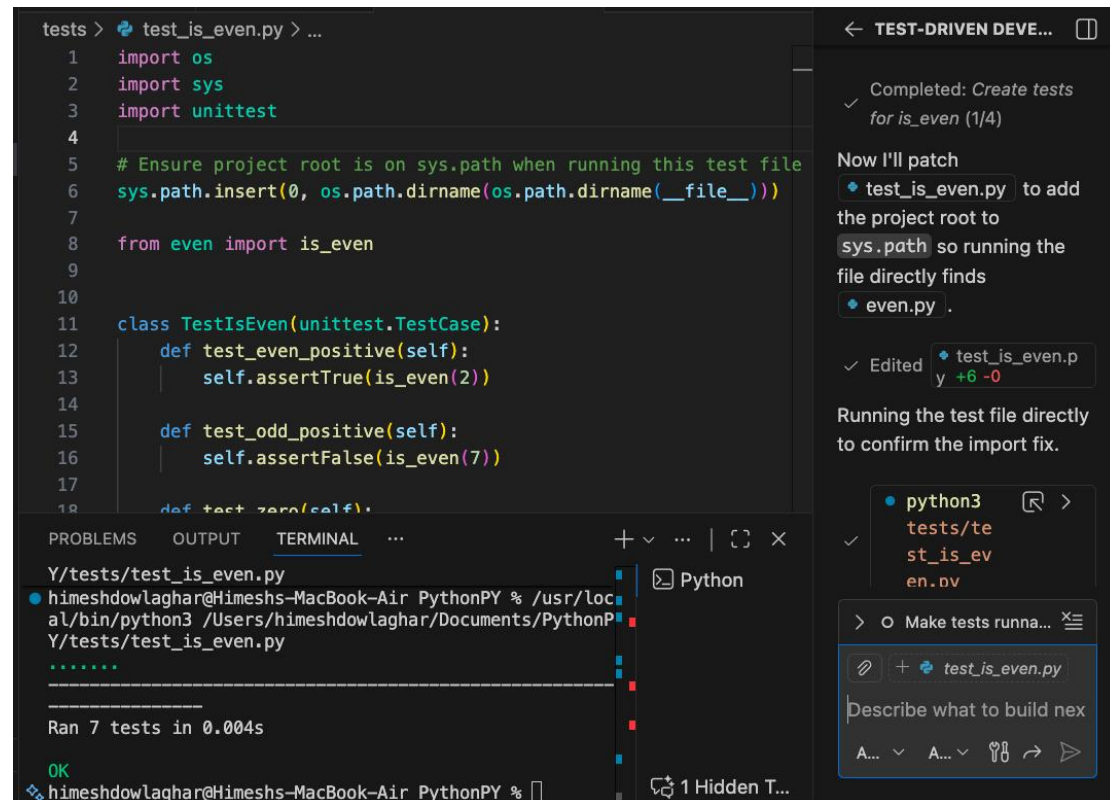
```

is_even(3.0)
with self.assertRaises(TypeError):
    is_even("2")

if __name__ == "__main__":
    unittest.main()

```

Proof of execution:



Task 2 – Test-Driven Development for String Case Converter

- Ask AI to generate test cases for two functions:

- `to_uppercase(text)`

- `to_lowercase(text)`

Requirements:

- Handle empty strings
- Handle mixed-case input
- Handle invalid inputs such as numbers or None

Example Test Scenarios:

`to_uppercase("ai coding")` → "AI CODING"

`to_lowercase("TEST")` → "test"

`to_uppercase("")` → ""

`to_lowercase(None)` → Error or safe handling

Code

```

def to_uppercase(text):
    """Return the uppercase form of `text`.

    Raises TypeError if input is not a str.

```

```

"""
if not isinstance(text, str):
    raise TypeError("Input must be a string")
return text.upper()

```

```

def to_lowercase(text):
    """Return the lowercase form of `text`.

```

Raises TypeError if input is not a str.

```

"""
if not isinstance(text, str):
    raise TypeError("Input must be a string")
return text.lower()

```

The screenshot shows a code editor with two Python functions: `to_uppercase` and `to_lowercase`. Both functions include docstrings and type error handling. The `to_uppercase` function is defined on lines 1-9, and `to_lowercase` is defined on lines 11-18. The code is being tested using a test runner interface on the right side of the editor. The test runner shows that all tests passed (13 total). It lists the files added: `strings.py` and `test_string_case.py`. It also provides next steps: commit changes, add a README, or extend the API to accept other types (e.g., bytes). The terminal at the bottom shows the command `python3 /Users/himeshdowlaghar/Documents/PythonPY/test_string_case.py` being executed, resulting in `OK`.

Task-3:

Test-Driven Development for List Sum Calculator

- Use AI to generate test cases for a function `sum_list(numbers)` that calculates the sum of list elements.

Requirements:

- Handle empty lists
- Handle negative numbers
- Ignore or safely handle non-numeric values

Example Test Scenarios:

`sum_list([1, 2, 3]) → 6`

`sum_list([]) → 0`

sum_list([-1, 5, -4]) → 0

sum_list([2, "a", 3]) → 5

Expected Output 3

- A robust list-sum function validated using AI-generated test cases.

Proof of execution:

The screenshot shows a code editor with a Python test file and a terminal window. The test file, `test_sum_list.py`, contains the following code:

```
1 import os
2 import sys
3 import unittest
4
5 # Ensure project root is on sys.path when running this test file
6 sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
7
8 from list_sum import sum_list
9
10
11 class TestSumList(unittest.TestCase):
12     def test_sum_basic(self):
13         self.assertEqual(sum_list([1, 2, 3]), 6)
14
15     def test_empty_list(self):
16         self.assertEqual(sum_list([]), 0)
17
18     def test_negative_numbers(self):
19         self.assertEqual(sum_list([-1, 5, -4]), 0)
```

The terminal window shows the execution of the tests:

```
Y/list_sum.py
himeshdowlaghar@Himeshs-MacBook-Air PythonPY % /usr/local/bin/python3 /Users/himeshdowlaghar/Documents/PythonPY/tests/test_sum_list.py
.....
Ran 7 tests in 0.004s
OK
```

On the right side of the editor, there is a sidebar with the following text:

TEST-DRIVEN DEVELOPMENT

Done — robust `sum_list` implementation and tests added, all tests pass.

- Files added:
 - `list_sum.py`,
 - `test_sum_list.py`
- Tests: run with:

```
python3 -m unittest di
```

Next steps: commit changes, add a README, or extend `sum_list` to accept other iterables — which would you like?

Todos (10/10)

- `test_sum_list.py`

Describe what to build next

A... A... A... A...

Task 4 – Test Cases for Student Result Class

- Generate test cases for a `StudentResult` class with the following methods:

- `add_marks(mark)`
- `calculate_average()`
- `get_result()`

Requirements:

- Marks must be between 0 and 100
- Average ≥ 40 → Pass, otherwise Fail

Example Test Scenarios:

Marks: [60, 70, 80] → Average: 70 → Result: Pass

Marks: [30, 35, 40] → Average: 35 → Result: Fail

Marks: [-10] → Error

Expected Output -4

- A fully functional `StudentResult` class that passes all AI-generated test

Code:

```
import os
import sys
import unittest
```

```
# Ensure project root is on sys.path when running this
test file directly
sys.path.insert(0,
os.path.dirname(os.path.dirname(__file__)))
```

```
from student_result import StudentResult
```

```
class TestStudentResult(unittest.TestCase):
def test_average_and_pass(self):
s = StudentResult()
s.add_marks(60)
s.add_marks(70)
s.add_marks(80)
self.assertAlmostEqual(s.calculate_average(), 70.0)
self.assertEqual(s.get_result(), "Pass")
```

```
def test_average_and_fail(self):
s = StudentResult()
s.add_marks(30)
s.add_marks(35)
s.add_marks(40)
self.assertAlmostEqual(s.calculate_average(), 35.0)
self.assertEqual(s.get_result(), "Fail")
```

```
def test_negative_mark_raises(self):
s = StudentResult()
with self.assertRaises(ValueError):
s.add_marks(-10)
```

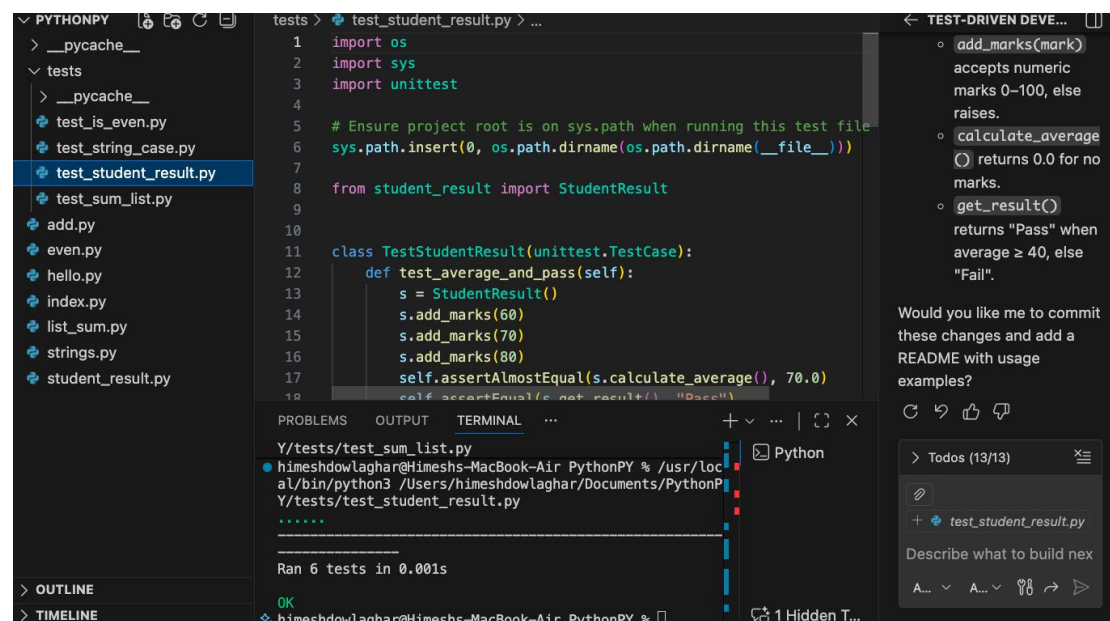
```
def test_mark_above_100_raises(self):
s = StudentResult()
with self.assertRaises(ValueError):
s.add_marks(150)
```

```
def test_non_numeric_raises(self):
s = StudentResult()
with self.assertRaises(TypeError):
s.add_marks(None)
```

```
def test_empty_marks(self):
s = StudentResult()
self.assertEqual(s.calculate_average(), 0.0)
self.assertEqual(s.get_result(), "Fail")
```

```
if __name__ == "__main__":
    unittest.main()
```

Proof of execution



Task-5

Task 5 – Test-Driven Development for Username Validator

Requirements:

- Minimum length: 5 characters
- No spaces allowed
- Only alphanumeric characters

Example Test Scenarios:

is_valid_username("user01") → True

is_valid_username("ai") → False

is_valid_username("user name") → False

is_valid_username("user@123") → False

Expected Output 5

A username validation function that passes all AI-generated test cases.

Code

```
def is_valid_username(username):
    """Return True if `username` is valid per rules:
    - Must be a string
```

- Minimum length 5
- No spaces
- Only alphanumeric characters

```
Raises TypeError if input is not a string.
"""
if not isinstance(username, str):
    raise TypeError("Username must be a string")
if len(username) < 5:
    return False
if " " in username:
    return False
if not username.isalnum():
    return False
return True
```

Proof of execution

The screenshot displays the Visual Studio Code editor with the `username_validator.py` file open. The code defines the `is_valid_username` function with docstrings and logic to validate usernames based on the specified rules. The interface includes a sidebar on the right showing the 'TEST-DRIVEN DEVELOPMENT' view with a test case for `test_username_validator.py`. The bottom panel shows the 'TERMINAL' output, which includes the command to run the tests and the successful execution results.

```
username_validator.py > is_valid_username
1 def is_valid_username(username):
2     """Return True if 'username' is valid per rules:
3
4     - Must be a string
5     - Minimum length 5
6     - No spaces
7     - Only alphanumeric characters
8
9     Raises TypeError if input is not a string.
10    """
11    if not isinstance(username, str):
12        raise TypeError("Username must be a string")
13    if len(username) < 5:
14        return False
15    if " " in username:
16        return False
17    if not username.isalnum():
18        return False
19    return True
```

TEST-DRIVEN DEVELOPMENT

- test_username_validator.py
- Behavior:
 - `is_valid_username(username)` raises `TypeError` for non-strings, returns True for valid usernames (length ≥ 5 , no spaces, alphanumeric), False otherwise.

Would you like these changes committed, or a short README showing usage and test commands?

TERMINAL

```
himeshdowlaghar@Himeshs-MacBook-Air PythonPY % /usr/local/bin/python3 /Users/himeshdowlaghar/Documents/PythonPY/tests/test_student_result.py
OK
himeshdowlaghar@Himeshs-MacBook-Air PythonPY % /usr/local/bin/python3 /Users/himeshdowlaghar/Documents/PythonPY/username_validator.py
himeshdowlaghar@Himeshs-MacBook-Air PythonPY % /usr/local/bin/python3 /Users/himeshdowlaghar/Documents/PythonPY/username_validator.py
himeshdowlaghar@Himeshs-MacBook-Air PythonPY %
```